

AGILE：一个全新的 LLM 智能体强化学习框架

Peiyuan Feng, Yichen He, Guanhua Huang, Yuan Lin,
Hanchong Zhang, Yuchen Zhang, Hang Li

ByteDance Research（字节跳动研究院） University of Science and Technology of China
Shanghai Jiao Tong University

英文原标题： AGILE: A Novel Reinforcement Learning Framework of LLM Agents

发表会议： NeurIPS 2024（第 38 届神经信息处理系统大会） **引用数：** 49 (Semantic Scholar, 2026-06-07)

代码与数据： <https://github.com/bytarnish/AGILE>

一句话定位： 把” 造一个智能体（agent）” 这件事整体当成一个**强化学习问题**来端到端训练——让大模型自己学会” 什么时候自己干、什么时候向人类专家求助”，结果用 7B/13B 开源小模型训练出的智能体反超了用 GPT-4 搭的智能体。

导读说明

本篇是面向 AI 应用开发实习生的**中文精读导读**（转化性学习材料），以讲解、转述和关键句翻译为主，帮助你快速读懂这篇论文的核心思想，并非逐字全文翻译。建议配合原文与代码仓库一起看。术语首次出现时括注英文原文，模型/数据集/方法专名（如 PPO、GPT-4、ProductQA）保持英文原样。

1 小白知识卡：读这篇前要懂的背景

LLM 智能体 (LLM agent)

就是” 以大语言模型为大脑” 的自动化程序。它不只会聊天，还能调用工具、查资料、记笔记、反思、做多步决策来完成一个完整任务（比如自动回答顾客的电商问题）。本文研究的就是怎么把这样一个智能体训得更好。

强化学习 (Reinforcement Learning, RL) 与策略 (policy)

一种” 试错学习” 范式：智能体在环境里做**动作** (action)，环境给**奖励** (reward)，目标是学会一套**策略**——即” 在什么状态下该做什么动作”——使长期累计奖励最大。本文最核心的一招：直接**把大模型本身当成这个策略**，模型每吐出一个 token 就算做了一个动作。

PPO (Proximal Policy Optimization, 近端策略优化)

目前训练 LLM 最常用的强化学习算法之一 (ChatGPT 的 RLHF 就用它)。直觉上: 让模型朝”能拿更高奖励”的方向更新参数, 同时用一个”近端约束”防止一步更新迈得太大、把模型练崩。本文用 PPO 来端到端优化整个智能体。

马尔可夫决策过程 (Markov Decision Process, MDP) 与价值函数 (value function)

MDP 是强化学习的数学框架, 由”状态、动作、状态转移、奖励”四要素组成。**价值函数** $V(s)$ 估计”从状态 s 出发, 未来预计能拿到多少总奖励”, 是判断一个状态好坏的标尺。后文的”求助是否划算”就靠它来衡量。

模仿学习 / SFT (Imitation Learning / Supervised Fine-Tuning)

先用”标准答案轨迹”(人类专家或更强的模型演示的操作序列)去监督训练模型, 让它先学会基本套路。本文是**两阶段训练**: 第一阶段 SFT 打底, 第二阶段再用 PPO 强化提升。

反思 (reflection) 与记忆 (memory)

反思: 把一次经历提炼成一条可复用的”知识/经验”。**记忆**: 把这些知识存起来, 下次遇到类似问题先去翻一翻。两者合起来让智能体能”越用越聪明”、把这次学到的东西用到以后。

主动求助 (seeking advice / proactive human-agent interaction)

本文提出的新能力: 当智能体觉得自己**没把握**时, 主动开口向人类专家(或更强的模型)要答案。难点在于”何时该问”——问多了浪费人力、问少了答错——这正好可以用 RL 的奖励来权衡并学出来。

2 摘要全译

我们提出了一个全新的 LLM 智能体强化学习框架, 命名为 AGILE (AGent that Interacts and Learns from Environments, 意为”与环境交互并从中学习的智能体”), 用于和用户完成复杂的对话式任务, 它综合利用了大语言模型 (LLM)、记忆 (memory)、工具 (tools) 以及与专家的交互。该智能体具备超越单纯对话的能力, 包括反思 (reflection)、工具使用 (tool usage) 和咨询专家 (expert consultation)。我们把构建这样一个 LLM 智能体的过程, 形式化为一个强化学习 (RL) 问题, 其中 LLM 充当策略模型 (policy model)。我们使用带有动作标注的数据和 PPO 算法对 LLM 进行微调。我们聚焦于问答任务, 并发布了一个面向智能体的数据集 ProductQA, 它由网络购物中具有挑战性的问题构成。我们在 ProductQA、MedMCQA 和 HotPotQA 上的大量实验表明: 基于 7B 和 13B 大模型、用 PPO 训练的 AGILE 智能体, 能够超越基于 GPT-4 的智能体。我们的消融实验 (ablation study) 突显了记忆、工具、咨询、反思和强化学习对于实现智能体强大性能而言都是不可或缺的。数据集与代码已开源: <https://github.com/bytarnish/AGILE>。

3 逐节精读

3.1 引言 (Introduction): 问题出在哪?

大模型已经展现出指令遵循、推理、零样本学习等能力,这催生了大量“LLM 智能体”的研究。近年的工作给智能体加了各种零件或工作流,比如**规划** (planning)、**反思** (reflection)、**工具使用** (tool-use)、**终身学习** (life-long learning)。

但论文指出一个关键空白: 这些零件目前是各搞各的,“如何把所有组件整合进一个统一框架、并端到端地一起优化,仍然不清楚”。

AGILE 的答案是: 用一个强化学习框架把所有东西串起来。如图 1 所示,智能体系统由四个模块组成: **LLM**、**记忆**、**工具**、**执行器** (executor); 此外它还能和**用户与专家交互**。其中:

- **LLM** 是“所有动作的预测者”: 它生成指令、处理回复——本质上就是 RL 里的策略。
- **执行器 (executor)** 是“所有动作的控制者”: 它把 LLM 发出的指令解释成具体操作 (去激活相应模块), 再把模块的返回结果喂回给 LLM。比如执行器可以从记忆里取一段文本拼到 LLM 的上下文里, 或把上下文里的一段摘录写进记忆。

论文还提出了一个新能力——**主动求助 (seeking advice)**: 智能体遇到自己解决不了的问题时, 会主动咨询人类专家; 并能对专家反馈做反思、记进记忆里供以后使用。最后, 论文提出一种基于 RL 的训练方法, **同时训练**“调用各模块的策略”和“推理、规划、反思、求助”等能力, 整个过程是端到端的。

虽然框架是通用的, 本文在**复杂问答 (QA)** 场景下做评测, 因为这是 LLM 智能体最有可能胜过“单用一个 LLM”的任务。但现有 QA 基准只针对能力的某个子集 (如只测反思、或只测记忆检索), 无法同时考察智能体“把所有模块和能力组合起来”的本事。为此作者造了新基准 **ProductQA** (详见第 3.4 节)。

关键数字 (引言里的预告)。 在 ProductQA 上用 Vicuna-13b 两阶段训练得到 agile-vic13b-ppo, 总分 (total score) 相对 GPT-4 提升 9.2%、相对 GPT-3.5 提升 90.8%。在 MedMCQA 上把基座模型准确率从 53.4% 提到 **85.2%** (在 31.6% 的题上求助), 超过了 GPT4-MedPrompt 的 79.1%。在 HotPotQA 上达到 67.5% 准确率, 远超最强基线的 48.2%。

本文的三大贡献。

1. 提出一个全新的 LLM 智能体 RL 框架, 支持端到端学习; 尤其让智能体能**主动向人类专家求助**, 带来两个好处: (1) 处理复杂难题时保证较高准确率; (2) 促成“向人类学习”, 从而提升对新任务的适应能力。
2. 开发了基准 ProductQA, 用于全面评估智能体在复杂问答上的能力。

3. 在多个任务上做实验，证明基于 13B/7B 开源模型、用 PPO 训练的 AGILE 智能体能超越 GPT-4 智能体。

3.2 方法：把”做智能体”写成一道强化学习题

3.2.1 智能体的 RL 形式化 (2.1)

这是全文最核心的建模思想，务必理解：

- 这是一个 **token 级别的 MDP**。动作空间 A 就是 LLM 的词表 (vocabulary)，**每个 token 就是一个动作**。因此 LLM 自然就是策略模型。
- **状态 (state)** 是一个二元组 (上下文 context, 记忆 memory)。
- **状态转移由执行器实现**。LLM 预测出一个新动作 a_t (一个新 token) 后，把控制权交给执行器；执行器按预定逻辑把当前状态 s_t 转移到下一状态 s_{t+1} ，再把控制权交还 LLM 去预测下一个动作。同时环境给出奖励 $r(s_t, a_t)$ 。

执行器具体怎么”转移状态”？对每个动作，它先把 token 拼到上下文末尾；然后检查一张**已注册的函数表**——如果这个 token 恰好是某个函数名，就执行对应函数（涉及记忆读写、工具调用、与环境交互），进一步改变状态。举几个例子：token 是 [GetQuestion] 就向用户要一个新问题并拼进上下文；token 是 [UpdateMemory] 就把上下文里某段写进记忆；token 是 [ClearContext] 就把上下文重置成 [BOS]。一句话：**LLM 通过”预测函数名”来操控记忆和工具，靠执行器去真正执行**。完整函数表见表 ??，运行实例见图 1(b)。

3.2.2 策略学习 (2.2)：损失只算在”动作 token”上

作者把策略学习当成”训练一个语言模型”来做。给定一条智能体轨迹 $\tau = (s_1, a_1, \dots, s_n, a_n)$ ，推导出一个训练序列 $(e_1, \dots, e_n)$ ：其中 e_i 是执行器在第 i 步拼进上下文的所有 token。如果 a_i 是个函数名 token，那么 e_i 就是 a_i 再加上”函数执行后追加的额外 token”的拼接；否则 $e_i = a_i$ 。每个 e_i 的**第一个 token** $\{a_1, \dots, a_n\}$ 被称为**动作 token (action token)**。

两个关键工程细节（这是把 LLM 当策略训练时最容易踩错的点）：

1. **损失只在动作 token 上计算**（执行器塞进来的环境文本不该让模型去”背诵”，所以不算损失）。
2. 用 c_i 作为 e_i 中各 token 的**注意力掩码 (attention mask)** —— c_i 是 LLM 在第 i 步实际看到的上下文，它可能比 $(e_1, \dots, e_{i-1})$ 短，因为执行器会删掉一些上下文 token。这保证训练时”模型看到的上下文”和它当初做决策时一致。

在**模仿学习 (IL)** 阶段：通过观察人类专家或更强智能体来生成轨迹，再据此微调 LLM。在**强化学习 (RL)** 阶段：把 LLM 当策略采样轨迹，给单个动作 token 赋奖励，然后用 PPO 之类的策略梯度方法优化——同样只更新动作 token、同样用 c_i 当注意力掩码。

长轨迹怎么办？ 一条轨迹可能长达数百万 token，没法直接训。作者利用轨迹的结构把它切成若干小段 (session, 会话)，比如每个 QA 会话开头都重置上下文，就按会话边界来切。但麻烦在于：各会话并不完全独立——早先会话里的动作会改写记忆，对后续会话产生长程影响 (long-range dependency)。为解决这个“跨会话依赖”，作者在附录 A 提出了一个会话级优化算法（见本导读第 3.8 节）。

3.2.3 与人类专家的交互 (2.3)：求助为什么天然适合用 RL 学

智能体可以调用 [SeekAdvice] 主动求助。它的价值有两点：(1) 当置信度低时直接要正确答案，保证应用所需的准确率；(2) 可以先用 [Reflection] 把专家建议提炼成通用知识再存进记忆，这种知识积累让它能适应训练时没见过的新任务。

为什么求助决策难、又为什么适合 RL？ 智能体得同时估计：自己当前的置信度、这条建议对未来会话的潜在价值、以及占用人力的成本。这个最优权衡很难人工标注，但和 RL 框架天然契合：当下风险、未来价值、动作成本都能表示成 RL 奖励，于是“求助”这项技能就能作为策略模型的一部分被端到端训练出来。

3.3 方法图解

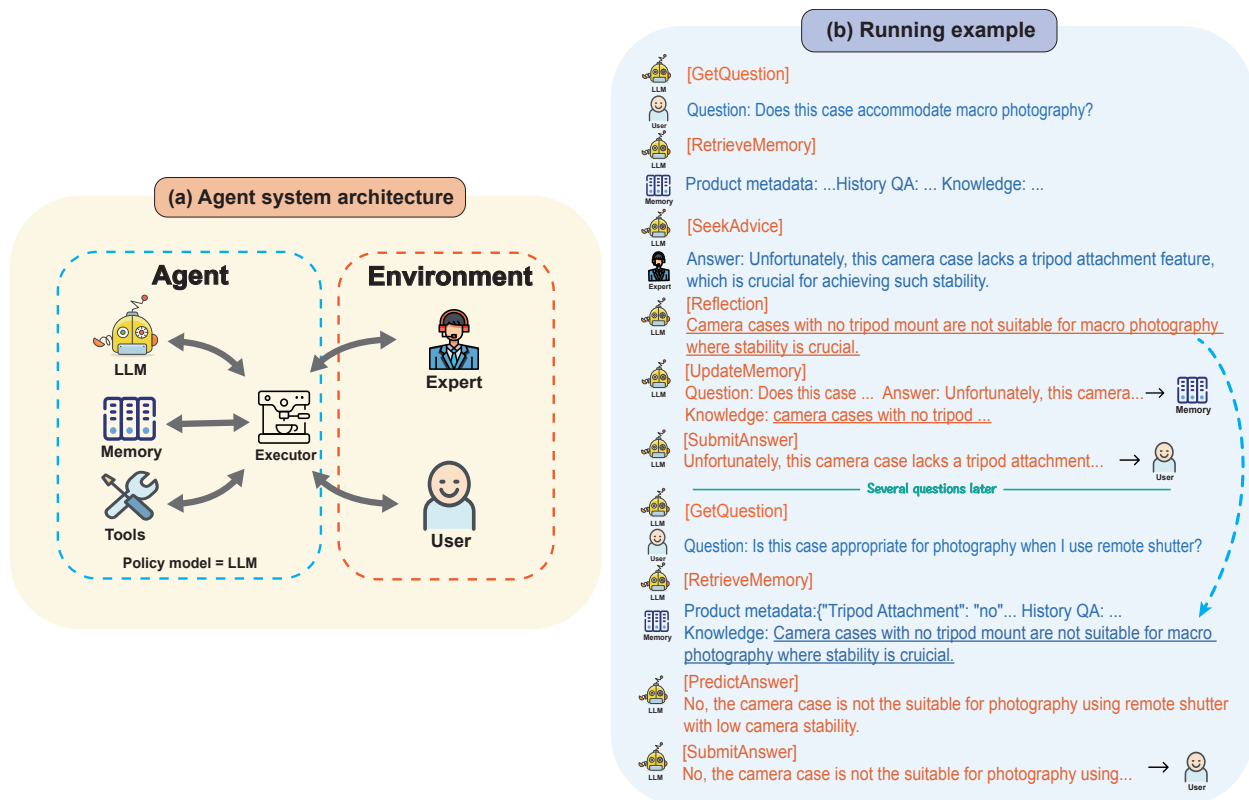


图 1: (a) 智能体系统架构，包含 LLM、记忆、工具、执行器 (executor)。(b) AGILE 在电商客服问答场景下的一个运行实例。LLM 生成的 token（即动作）用橙色标注，执行器追加的 token 用蓝色标注。

这张图在说什么、数据怎么流动。看左半张图 1(a): 左边虚线框是**智能体 (Agent)** 内部——LLM、Memory、Tools 三个模块都通过中间的**执行器 (Executor)** 来联动; 右边虚线框是**环境 (Environment)**——包含 Expert (专家) 和 User (用户)。注意图底那行小字 **"Policy model = LLM"**: 这就是全文的灵魂——大模型本身就是被训练的策略。所有箭头都汇聚到执行器, 说明执行器是“总调度”: 它把 LLM 的指令翻译成对记忆/工具/环境的实际操作, 再把结果送回 LLM。

右半张图 1(b) 是一次真实运行, 把数据流走一遍就懂了。第一次遇到“这个相机包适合微距摄影吗?”: LLM 先 [GetQuestion] 取问题 → [RetrieveMemory] 翻记忆 (这时还没相关知识) → 发现没把握, 于是 [SeekAdvice] 向专家求助拿到答案 → [Reflection] 把它提炼成一条通用知识“没有三脚架接口的相机包不适合需要稳定性的微距摄影” → [UpdateMemory] 把问答和知识都写进记忆 → [SubmitAnswer] 回复用户。**几个问题之后**, 再遇到类似的“用遥控快门时这个包适合摄影吗?”, 这次 [RetrieveMemory] 就能从记忆里翻出上次那条知识, 于是直接 [PredictAnswer] 自己答出来、不用再求助。**这就是“求助 → 反思 → 记忆 → 下次自给自足”的闭环**, 也是智能体“越用越独立”的来源。

3.4 ProductQA 数据集 (第 3 节)

作者认为“真实网购环境下的商品问答”是评测 LLM 智能体的绝佳综合挑战, 因为它同时要求: (1) 关于数百万商品的专家知识 (技术参数、特定场景用法、与其他产品的兼容性); (2) 会用工具 (如商品搜索); (3) 不断有新商品出现, 要求智能体有适应性。和已有的网购问答数据集 (主要问商品元数据/页面信息) 不同, ProductQA 包含更复杂的查询, 涉及推理、专家知识和工具使用 (如 SQL)。

规模与构成: 26 个 QA 任务, 每个对应一个 Amazon 商品类目 (如耳机、主板、相机包); 其中 20 个类目用于训练、6 个用于测试 (训练/测试类目**互不重叠**, 专门考察“对新任务的适应力”)。每个商品组平均约 3,393 个问答对, 总计 88,229 对。由 20 名至少大学学历的专业标注员按市场价标注。

三类问题 (见表 1):

- **Fact-QA:** 关于具体商品细节的事实题 (从商品评论中构造)。
- **Search-QA:** 按用户偏好做商品推荐的检索题 (先用规则生成随机 SQL, 再由 GPT-4 翻译成自然语言问题, 答案通过执行 SQL 得到)。
- **Reasoning-QA:** 需要领域推理的题, 比如“某个商品特性意味着什么” (先收集专业知识条目, 再据此出题)。

每题都标了“一段话的详细长答案 (像人工客服)”和“几个词的短答案”。**长答案准确率用 GPT-4 评判, 短答案用精确匹配 (exact match) 评判、并用来定义 RL 的奖励。**

表 1: ProductQA 中 Fact-QA、Search-QA、Reasoning-QA 的示例（节选自表 3）。

类型	问题（示例）	短答案
Fact-QA	JVC HA-EB75 耳机用的钹磁单元尺寸是多少？	13.5 mm
Search-QA	我是个总在路上的发烧友，想要续航最长的耳机（入耳/耳挂均可），有哪款？	B00LJT2EPK
Reasoning-QA	我用来剪辑音乐时，这款耳机的音质能和有线耳机相当吗？	no

3.5 实验（第 4 节）

实验设置。 在三个复杂 QA 任务上评测：**ProductQA**、**MedMCQA**（医学院入学考的选择题）、**HotPotQA**（自然的多跳问题，考推理 + 搜索工具）。基座模型：ProductQA 和 HotPotQA 用 Vicuna-13b-1.5；MedMCQA 用医学模型 Meerkat-7b。训练两阶段：先模仿学习，再用算法 1 做 RL。模型全参数训练，没用 LoRA，跑在 NVIDIA-H800 上。

三个评测指标（要记住）。

- **Advice rate（求助率）**：求助的比例——越低越好（少占人力）。
- **Accuracy（准确率）**：答对的比例——越高越好。
- **Total score（总分）**：所有会话的平均奖励，**同时**把求助率和准确率算进去。奖励规则：答对 +1，答错 0，求助固定扣 $-c$ （ c 是求助成本）。所以可能的总奖励只有 $\{0, 1, 1 - c\}$ 三种。这个总分是核心指标，因为它惩罚了“靠狂问人类来刷准确率”。

对比的基线。 两类：(1) 直接 prompt GPT-3.5 / GPT-4 答题（不以智能体方式工作），记作 gpt3.5-prompt / gpt4-prompt；(2) 把 **GPT-3.5 / GPT-4 也放进同一个 AGILE 框架里**（记作 agile-gpt3.5-prompt / agile-gpt4-prompt）——这是**公平对比**的关键：让强模型也能用记忆、工具、求助，看开源小模型 + RL 能不能反超。

3.5.1 ProductQA 结果

表 2: ProductQA 主结果（重排自表 4，6 个测试任务平均，求助成本 $c = 0.3$ ）。↑ 越高越好，↓ 越低越好。

方法	求助率 ↓ Advice Rate	准确率 ↑		总分 ↑	
		短答案	长答案	短答案	长答案
gpt3.5-prompt	-	0.202	0.322	-	-
gpt4-prompt	-	0.464	0.571	-	-
agile-vic13b-prompt	0.174	0.174	0.294	0.122	0.242
agile-gpt3.5-prompt	0.323	0.508	0.644	0.411	0.547
agile-gpt4-prompt	0.208	0.780	0.809	0.718	0.747
agile-vic7b-ppo (本文)	0.179	0.818	0.800	0.764	0.746
agile-vic13b-ppo (本文)	0.233	0.854	0.854	0.784	0.784

这些数字说明了什么：

- agile-vic13b-ppo 总分全面领先：短答案总分 0.784 相对 agile-gpt4-prompt (0.718) **相对提升 9.2%**；用差不多的求助次数，短答案准确率高出 7.4%。
- 连更小的 agile-vic7b-ppo (7B) 的总分都超过了用 GPT-4 的智能体。注意：GPT-4 智能体的 prompt 里是告知了求助成本的，对比是公平的。
- 对比 agile-vic13b-prompt（同样 13B 但不训练、纯 prompt，总分仅 0.122）可见：**光把开源模型塞进框架远远不够，RL 训练才是性能的来源。**
- 调节求助成本 c 能灵活控制”准确率 vs. 人力成本”的权衡： $c = 0.5$ 时几乎不求助； $c = 0.1$ 时准确率接近 1（如主板任务可达 94.1%）。这对”要求极高准确率”的真实场景很重要。

消融实验（表 5，证明每个零件都不可或缺）。以 agile-vic13b-ppo（总分 0.784）为满配基准，逐一拆掉某个能力：

表 3: ProductQA 消融实验（重排自表 5）。括号内为总分相对满配的变化。

方法	求助率 ↓	准确率 ↑	总分 ↑
w/o Reflection（去掉反思）	0.270	0.852	0.771 (-1.7%)
w/o Memory（去掉记忆）	0.407	0.876	0.754 (-4.0%)
w/o Advice（去掉求助）	0.000	0.747	0.747 (-5.0%)
non-adapt-advice（固定时机求助）	0.233	0.812	0.742 (-5.7%)
w/o Tool-Use（去掉工具）	0.492	0.864	0.717 (-9.3%)
w/o RL（只 SFT、不做 RL）	0.256	0.843	0.766 (-2.3%)
agile-vic13b-ppo (满配)	0.233	0.854	0.784

读这张表的窍门：**拆掉记忆或工具后，求助率会飙升**（记忆 0.233→0.407，工具 0.233→0.492），因为智能体“自己变弱了、只好更频繁地求人”——这正好反证了记忆和工具的价值。其中：去掉求助直接掉 10.7% 准确率；把求助改成“固定在轨迹开头问”（non-adapt）准确率掉 4.2%，说明“**自适应地决定何时求助**”本身很有价值；RL 训练相对 SFT 再提 2.3% 总分、同时降低求助率、提高准确率。另外图 4 显示：随着会话变多，agile-vic13b-ppo 的求助率**持续下降**（越用越独立），而去掉 RL 或反思都会让求助率显著上升。

3.5.2 MedMCQA 与 HotPotQA 结果

MedMCQA (医学选择题)：agile-mek7b-ppo（基于小小的 7B Meerkat）达到 **85.2%** 准确率（求助率 31.6%），相比基座模型提升 31.8 个百分点，**比当时 SOTA 的 gpt4-Medprompt (79.1%) 还高 6.1%**。论文有个很有意思的发现：单“会求助”这一项就带来 23.2% 的准确率提升，意味着**平均每次求助能纠正 0.73 个预测错误**——说明 PPO 训练让模型很会“自知之明”，专挑自己会错的题去问。在所有智能体都能求助的公平设定下，本文智能体总分也高于 GPT-4 智能体。

HotPotQA (多跳问答)：作者发现原版 ReAct 实现偏弱，自己用 GPT-4 复现 (ReAct-gpt4-prompt) 得到更强基线 48.2%。agile-vic13b-ppo 准确率 67.5%（精确匹配），**相对最强基线提升 40.0%**；相比 agile-gpt4-prompt 既准确率更高、求助率又更低，总分相对提升 10.8%。消融同样证明去掉求助或 RL 都会让总分显著下降。

3.6 相关工作（第 5 节）：AGILE 的独特之处

论文用一张对比表（表 8）把自己和 WebGPT、ReAct、Reflexion、ChatDev、RAP、AutoAct、TPTU 等放在一起比。结论一句话：**AGILE 是首个“用强化学习训练整个智能体、并内置主动向人类求助”的工作**——它同时勾齐了 SFT、RL、Memory、Tools、Reflection、以及“主动的人机交互 (Proactive Human-agent Interaction)”这六项，而其他工作都缺其中至少一项（尤其“主动求助”几乎只有 AGILE 有）。

关于**人机交互**，论文澄清了和已有工作的区别：以往要么是“被动接受反馈/按预定规则”，要么用行为克隆 (behavior cloning) 教模型提问、但忽略了“是否该求助应当取决于 LLM 自身的知识与能力”；也有人用校准过的 token 概率当置信度，但 token 概率往往**过度自信 (overconfident)**，且现有校准方法在“序列里要做多个决策”的智能体场景下泛化不好。**AGILE 的解法是把求助的价值与成本直接写成 RL 奖励**，于是“该不该问”作为策略的一部分被端到端学出来。

关于**基准**，论文用表 9 说明：现有基准 (Webshop、Mind2Web、ALFWorld、ScienceWorld、HotPotQA、TriviaQA) 没有任何一个能同时覆盖“长轨迹、工具使用、长期知识积累、跨任务”四项，而 **ProductQA 四项全占**，这正是造它的理由。

3.7 结论与未来工作（第 6 节）

总结三点：(1) AGILE 整个系统用 RL 端到端训练；(2) 具备向外部人类专家求助的能力；(3) 配套发布了有挑战性的复杂问答数据集 ProductQA。大量实验证明：在该框架下，**基于小模型、经 RL 训练的智能体可以超越 GPT-4**。

作者还展望：可以给智能体接更多工具（多模态感知、物理世界操作、逻辑推理等）。他们提出一个有意思的类比——AGILE 的活动可分两类：“只用 LLM 自己”对应人类认知的 **System 1**（快思考），“LLM + 工具”对应 **System 2**（慢思考）；而 AGILE 的记忆则是经验与知识的积累库，是自我提升的关键。框架还能扩展到师生、辩论、协作等多种人机/多智能体交互形式。

3.8 附录精华：会话级优化算法（附录 A）

这是解决前面“长轨迹 + 跨会话依赖”难题的数学工具，理解直觉即可。

把整条轨迹 τ 切成若干会话 $(\tau_1, \dots, \tau_n)$ 。总目标是最大化期望总奖励：

$$R(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [r(\tau)].$$

其中 π_θ 是参数为 θ 的策略， $r(\tau)$ 是轨迹总奖励。直接优化的难点：参数 θ 同时出现在“前段、本段、后段”三处期望里，互相纠缠。作者的办法是固定一个**基准策略** θ_k ，构造一个只在“本会话”里含 θ 的**近端目标** $R(\theta|\theta_k)$ （思路和 PPO 的“近端”一脉相承），从而可以**把优化拆到会话级别单独做**。

核心是定义一个**代理奖励**（proxy reward）（论文式 5）：

$$\tilde{r}_k(\tau_i) := r(\tau_i) + (V_{\pi_{\theta_k}}(S_{i+1}) - V_{\pi_{\theta_k}}(S_i)).$$

逐符号拆解： $r(\tau_i)$ 是会话 i 当下拿到的奖励； S_i 、 S_{i+1} 分别是会话 i 和 $i+1$ 的初始状态； $V_{\pi_{\theta_k}}(\cdot)$ 是基准策略下的价值函数。括号里这一项

$$A_i := V_{\pi_{\theta_k}}(S_{i+1}) - V_{\pi_{\theta_k}}(S_i)$$

被称为**状态优势函数**（state advantage function），它度量“经过会话 i 之后，状态从 S_i 变到 S_{i+1} 究竟变好了多少”。**直觉**：如果会话 i 里求助学到的新知识在后续会话有用， A_i 就该变大（鼓励这次求助）；如果记忆里早有大量类似知识， A_i 就该变小（这次求助是浪费）。这样就把“求助对未来的长期价值”折算进了**当前会话**的奖励里，使得每个会话可以被当成近似独立的样本、直接丢给 PPO 训练（算法 1）。论文还给出一个启发式定义（式 7），用“后续相似问题的数量 / 此前已入记忆的相似问题数 + 1”来估 A_i ，默认系数 $\beta = 0.1$ 。

4 核心 takeaways 与对 AI 应用开发的启示

1. “把整个 agent 当成一个 RL 问题端到端训”是一种范式转变。不再是手工拼接“规划 + 工具 + 反思”这些零件、各调各的 prompt，而是让 LLM 作为统一策略，用 PPO 把“何时调工具、何时查记忆、何时求助、怎么反思”一起学出来。这是本文最值得记住的思想。

2. ”会自知、会求助”是被严重低估的 agent 能力。让 agent 知道自己”不会”并主动求助，比硬答更有价值（MedMCQA 上每次求助平均纠错 0.73 个）。难点”何时该问”不要试图手写规则——用奖励（答对 +1 / 答错 0 / 求助 -c）让模型自己权衡，再用一个成本系数 c 当旋钮去调”准确率 vs. 人力成本”。

3. 小模型 + 好框架 + RL 能打过大模型 + 纯 prompt。7B/13B 开源模型经两阶段训练后总分超过把 GPT-4 放进同一框架的版本。对成本敏感的实际产品，这说明投入训练（而非一味堆大模型 API）可能更划算，且数据/权重可控。

4. ”求助 → 反思 → 记忆”闭环让 agent 越用越独立。实验里求助率随会话增多而持续下降。做产品时可以照搬这个闭环：把每次人工兜底的答案提炼成可复用知识写入知识库（记忆），下次相似问题就能自给自足，长期摊薄人力成本。

5. 工程细节决定成败。把 LLM 当策略训练时，记住两条：损失只算在”动作 token”上（别让模型去背环境塞进来的文本）；用真实历史上下文当注意力掩码（保证训练与决策时所见一致）。长轨迹要按会话切分，并用”状态优势/代理奖励”把跨会话的长期价值折算回当前会话。

6. 评测要用”总分”而非单看准确率。准确率可以靠狂问人类刷高，但那不实用。本文的 total score 同时惩罚求助成本，是更贴近落地的指标。设计自己的 agent 评测时，务必把”调用外部资源/人力的代价”算进总目标。

关键参考文献（节选，原文保留）

1. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017. (PPO 算法)
2. Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629, 2022. (HotPotQA 上的主要基线)
3. Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering. PMLR, 2022.
4. Zhilin Yang, Peng Qi, Saizheng Zhang, et al. HotPotQA: A dataset for diverse, explainable multi-hop question answering. arXiv preprint arXiv:1809.09600, 2018.
5. Wei-Lin Chiang, Zhuohan Li, Zi Lin, et al. Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality, 2023. (ProductQA/HotPotQA 的基座模型)
6. Hyunjae Kim, Hyeon Hwang, Jiwoo Lee, et al. Small language models learn enhanced reasoning skills from medical textbooks (Meerkat). arXiv preprint arXiv:2404.00376, 2024. (MedMCQA 的基座模型)

本导读为转化性学习材料，仅供学习交流。论文版权归原作者及 *NeurIPS 2024* 所有，原文与代码见
<https://github.com/bytarnish/AGILE>。